

Algorithm for file updates in Python

Project description

As a security professional at a healthcare company, I am responsible for maintaining strict access controls for systems containing sensitive patient records. Access to these restricted subnetworks is governed by an IP address allow list, which must be regularly updated to remove employees who no longer require access. To automate and secure this process, I developed a Python algorithm that parses the existing allow list, cross-references it against a designated remove list, and automatically purges unauthorized IP addresses. This script ensures our access controls remain accurate, compliant, and resilient against unauthorized access attempts.

Open the file that contains the allow list

To begin the process, I need to access the existing allow list stored in a text file named "allow_list.txt". I assign this filename to a variable called `import_file`. Then, I use Python's `with` statement combined with the `open()` function to safely open the file in read mode ("r"). The `with` keyword is crucial here because it acts as a context manager, ensuring the file is automatically and properly closed after the operations inside the block are completed, preventing resource leaks. I use the variable `file` to store the file object while working with it.

Screenshot:

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# First line of `with` statement
with open(import_file, "r") as file:
```

Note: Running the code above will produce an error because it only contains the first line of the `with` statement, It will be completed in the next step

Read the file contents

Once the file is open, I need to extract its contents so Python can analyze the data. I use the `.read()` method on the file object, which reads the entire text file and returns it as a single continuous string. I store this raw string data in a new variable named `ip_addresses`. At this stage, the data is still unstructured text, but it is now loaded into memory and ready for parsing.

Screenshot:

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`
    ip_addresses = file.read()

# Display `ip_addresses`
print(ip_addresses)
```

```
ip_address
192.168.25.60
192.168.205.12
192.168.97.225
192.168.6.9
```

Activate

Convert the string into a list

To manipulate individual IP addresses, the continuous string needs to be broken down into discrete, iterable elements. I apply the `.split()` method to the `ip_addresses` string. By default, `.split()` separates the string at whitespace (like spaces or newline characters) and returns a list of individual strings. I reassign the result back to the `ip_addresses` variable, effectively transforming the data from a single string into a Python list, which allows me to loop through and evaluate each IP address individually.

Screenshot:

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Display `ip_addresses`

print(ip_addresses)
```

['ip_address', '192.168.25.60', '192.168.205.12', '192.168.97.225', '192.168.6.9', '192.168.52.90', '192.168.158.170', '192.168.90.124', '192.168.186.176', '192.168.133.188', '192.168.203.198', '192.168.201.40', '192.168.218.219', '192.168.52.37', '192.168.156.224', '192.168.60.153', '192.168.58.57', '192.168.69.116']

Activate

Iterate through the remove list

To identify which IP addresses need to be purged, I set up a for loop to iterate through the list of IP addresses. I use `element` as the loop variable, which will temporarily hold each IP address from the allow list during each iteration. This iterative structure allows the algorithm to systematically check every IP address against the secondary remove list to see if it needs to be revoked.

Screenshot:

```
# Assign `import_file` to the name of the file

import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.

remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file

with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list

ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Display `element` in every iteration

    print(element)
```

ip_address
192.168.25.60
192.168.205.12
192.168.97.225
192.168.6.9

A
G

Remove IP addresses that are on the remove list

Inside the for loop, I nest an if conditional statement to check if the current `element` actually exists within the `remove_list` using the `in` operator. If the condition evaluates to `True`, I apply the `.remove()` method to the `ip_addresses` list, passing an `element` as the argument to delete that specific IP address. Applying the `.remove()` method in this way is possible because there are no duplicates in the `ip_addresses` list. This ensures that the script only attempts to remove valid, existing entries without throwing errors.

Screenshot:

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Display `ip_addresses`
print(ip_addresses)

['ip_address', '192.168.25.60', '192.168.205.12', '192.168.6.9', '192.168.52.90', '192.168.90.124', '192.168.186.176', '192.168.133.188', '192.168.203.198', '192.168.218.219', '192.168.52.37', '192.168.156.224', '192.168.60.153', '192.168.69.116']
```

Update the file with the revised list of IP addresses

After purging the unauthorized IPs from the list in memory, I need to save the updated allow list back to the text file. First, I convert the `ip_addresses` list back into a single string using the `.join()` method. I apply `.join()` to the string `"\n"` in order to separate the elements in the file by placing them on a new line, restoring the file's original formatting. Next, I use a second `with` statement to open the `import_file` again, this time in write mode (`"w"`). Inside this block, I use the `.write()` method to overwrite the file's old contents with the newly formatted, sanitized string.

Screenshot:

```
# Assign `import_file` to the name of the file
import_file = "allow_list.txt"

# Assign `remove_list` to a list of IP addresses that are no longer allowed to access restricted information.
remove_list = ["192.168.97.225", "192.168.158.170", "192.168.201.40", "192.168.58.57"]

# Build `with` statement to read in the initial contents of the file
with open(import_file, "r") as file:

    # Use `.read()` to read the imported file and store it in a variable named `ip_addresses`

    ip_addresses = file.read()

# Use `.split()` to convert `ip_addresses` from a string to a list
ip_addresses = ip_addresses.split()

# Build iterative statement
# Name loop variable `element`
# Loop through `ip_addresses`

for element in ip_addresses:

    # Build conditional statement
    # If current element is in `remove_list`,

    if element in remove_list:

        # then current element should be removed from `ip_addresses`

        ip_addresses.remove(element)

# Convert `ip_addresses` back to a string so that it can be written into the text file
ip_addresses = "\n".join(ip_addresses)

# Build `with` statement to rewrite the original file
with open(import_file, "w") as file:

    # Rewrite the file, replacing its contents with `ip_addresses`

    file.write(ip_addresses)
```

Summary

This Python algorithm provides a reliable, automated solution for maintaining secure access control lists in a healthcare environment. By leveraging file I/O operations like `open()`, `.read()`, and `.write()`, the script seamlessly interacts with external text files. The use of `.split()` and `.join()` effectively bridges the gap between unstructured text data and structured Python lists, enabling precise data manipulation. Furthermore, combining a `for` loop with an `if` conditional and the `.remove()` method ensures that only authorized IP addresses remain in the system. Ultimately, this automation reduces human error, ensures strict compliance with patient data access policies, and strengthens the organization's overall security posture.